

LMAO-World: Loco-Manipulation VLA Optimization inside a World Model

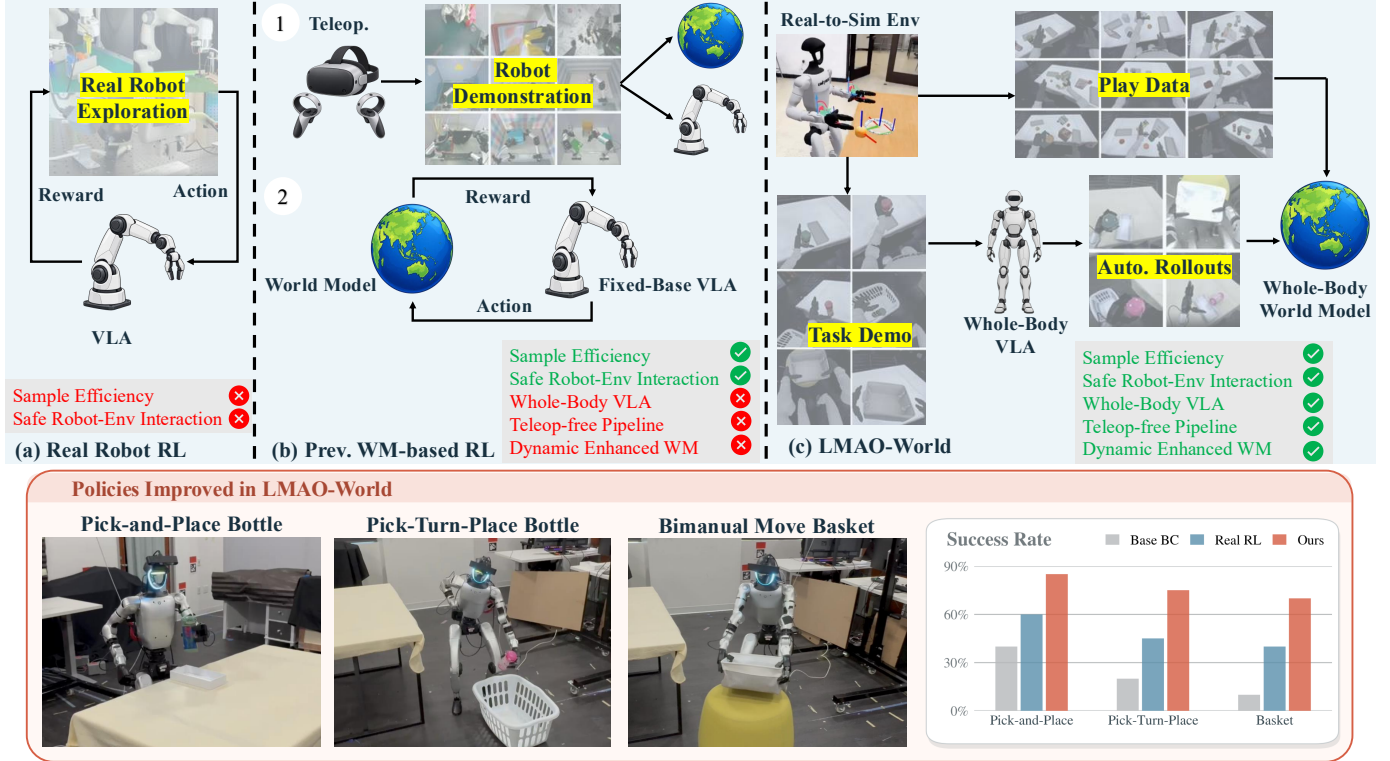


Fig. 1. **LMAO-World: teleoperation-free world model RL for humanoid loco-manipulation.** (a) Real-robot RL is costly and risky. (b) Prior world model RL focuses on fixed-base manipulation and uses teleoperation data. (c) LMAO-World trains a whole-body VLA from synthetic demonstrations, mid-trains the world model on simulated play data, and adapts it with autonomous real-robot rollouts. Policy improvement in the adapted world model forms a sim-to-dream-to-real flywheel; the bottom panel reports real-robot performance on three tasks.

Abstract—Reinforcement learning (RL) can improve vision-language-action (VLA) policies beyond imitation, but real-robot exploration is costly and unsafe for humanoid robots, while direct optimization in reconstructed simulators can amplify sim-to-real errors. We present LMAO-World, a teleoperation-free pipeline that post-trains a synthetic-data humanoid VLA inside an action-conditioned video world model (WM). A 3D Gaussian Splatting (3DGS) simulator [1] supplies task demonstrations and manipulation and locomotion play data; 20 autonomous real-robot rollouts per task adapt the WM to deployment observations. Policy rollouts then couple generated video with SONIC–MuJoCo dynamics and receive dense stage-aware progress rewards. Across three real-robot tasks, play data improves every WM metric and raises mean policy success from 63.3% to 75.0%. We instantiate AWR, ReinFlow, and DICE-RL, all of which outperform behavior cloning. DICE-RL reaches 75.0% mean success, compared with 23.3% for behavior cloning and 48.3% for real-robot RL. Ablations identify dense rewards, policy-dependent rollout refresh, and play data as complementary requirements. These results demonstrate WM-RL for whole-body humanoid loco-manipulation without target-task teleoperation.

Index Terms—World Models, Vision-Language-Action models,

Reinforcement Learning, Humanoid loco-manipulation.

I. INTRODUCTION

Vision-language-action (VLA) models [2]–[6] couple pre-trained vision-language backbones with action heads learned from demonstrations, whose scale and quality strongly determine performance. Collecting such data through teleoperation is particularly difficult for high-degree-of-freedom (DoF) humanoids. Whole-body controllers (WBCs) [7], [8] coordinate balance, locomotion, and bimanual control, but do not remove this bottleneck. Real-to-sim-to-real (R2S2R) systems such as the LEGS simulator [9] and VLK [10] instead reconstruct real scenes and generate whole-body trajectories without teleoperation. Although policies trained on these synthetic data can transfer to hardware, residual appearance and dynamics gaps require further learning from deployment experience.

Reinforcement learning can improve a behavior-cloned (BC) policy under compounding errors and residual sim-to-real mismatch, but VLA post-training requires long-horizon re-

wards and optimization through diffusion or flow action heads. Offline-to-online methods often update simple MLP policies [11]–[13], limiting expressiveness through their tractable action densities. Advantage-weighted methods support richer policies [14]–[17], but often require human DAgger interventions. RL-100 [18] differentiates the PPO objective [19] through the full denoising process at high computational cost. Steering [20], [21], residual methods [22], [23], and DICE-RL [24] adapt pretrained policy outputs more directly. Nevertheless, these approaches rely on continued real-robot interaction or fixed offline data, neither of which supports exploratory whole-body humanoid learning.

Learned *world models* (WMs) shift policy optimization away from hardware by predicting future observations under candidate actions. Existing systems use either compact dynamics representations or generative video models. Latent dynamics models and explicit 3D digital twins [25]–[28] enable sample-efficient learning, but are usually scene-specific and do not provide the open-world visual inputs required by VLAs. Generative video WMs instead adapt large pretrained models to robot actions. VLAW [29] treats successful WM rollouts as behavior-cloning targets, while WMPO [30] and related methods optimize policies on generated trajectories [31]–[34]. WoVR [35], WorldSample [36], and PlayWorld [37] improve rollout initialization, transition selection, and autonomous data collection, respectively. However, these methods remain limited to fixed-base tabletop manipulation and use human teleoperation data during training.

We study whether WM-RL can support humanoid locomotion VLA post-training. Starting from a VLA trained entirely on synthetic data, we use autonomous real-robot rollouts to adapt the WM and then improve the policy inside the WM. This produces the teleoperation-free flywheel as shown in Fig. 1. Our contributions are:

- 1) We build LMAO-World, a WM-RL system for post-training whole-body humanoid VLAs and, to our knowledge, the first to extend this paradigm beyond fixed-base tabletop manipulation. Its design addresses coupled locomotion and manipulation through policy adaptation across varying base poses and WM prediction of ego-centric observations under whole-body motion.
- 2) We develop an autonomous whole-body play data collection pipeline for mid-training the WM, improving its physical fidelity and broadening state-action coverage before task-specific fine-tuning.
- 3) We establish a teleoperation-free policy improvement flywheel by combining R2S2R pipeline with WM. Extensive experiments on three tasks with three RL algorithms evaluate its sample efficiency and applicability across policy optimizers.

Humanoid safety constraints determine our system design. Because unconstrained autonomous play is impractical on hardware, we collect play data in simulation and optimize the policy in a learned WM. Unlike a 3DGS-based simulator, whose residual sim-to-real gap can be exploited by RL, the WM is adapted directly with autonomous real-robot rollouts.

Simulated play data complement this adaptation with physically grounded contacts and broader whole-body state-action coverage, reducing extrapolation as policy improvement moves beyond the current policy distribution.

II. BACKGROUND

A. The LEGS Simulator

The LEGS simulator [9] is an R2S2R data engine for the Unitree G1. It combines a 3DGS reconstruction of the static scene with explicit textured object meshes recovered using SAM 3D [38]. MuJoCo [39] simulates robot-object contacts, while the 3DGS scene provides background rendering. Scripted locomotion and manipulation primitives are tracked by the SONIC WBC [7], producing teleoperation-free trajectories with state, contact, and action annotations. We use these trajectories to train the base VLA and to adapt the WM with play data and task-specific data.

B. Dense Progress Reward Models

SARM [40] maps a video and task instruction to dense task progress. A SigLIP-2 backbone [41] predicts a task stage s_t and within-stage completion ρ_t , giving $r_t = s_t + \rho_t$. The executed primitive supplies stage labels for trajectories from the LEGS simulator. Following SARM2 [42], successful labels increase monotonically, whereas failed trajectories reset to zero at the detected failure. Fine-tuning on autonomous real rollouts adapts the model to deployment observations.

C. DreamDojo: A Whole-Body-Control World Model

DreamDojo [43] is an action-conditioned video WM built on Cosmos [44]. Given an observation history and achieved 43-DoF joint states, it predicts future frames in fixed-length chunks, $\hat{o}_{t+1:t+k} = W_\phi(o_{\leq t}, a_{t:t+k})$. Its post-training on real Unitree G1 WBC trajectories provides a whole-body prior, but rollout generation must capture locomotion-induced viewpoint changes, self-occlusion, and contacts. Autoregressive error accumulation [45]–[47] motivates broad play data and bounded rollout horizons.

D. Reinforcement Learning for Flow-Matching Policies

A flow-matching action head generates an action chunk by integrating a learned velocity field, but its implicit likelihood complicates standard policy gradients. We consider three alternatives. **DICE-RL** [24] searches modes of a frozen flow prior π_{pre} and learns a local residual correction:

$$\begin{aligned} a_i &= \underbrace{\pi_{\text{pre}}(o, z_i)}_{\text{frozen prior}} + r_\psi(o, z_i), \\ z^* &= \arg \max_{i \leq K} \min_{j \leq M} Q_j(o, z_i, a_i), \end{aligned} \quad (1)$$

AWR [16] performs advantage-weighted supervised regression:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{(o,a) \sim \mathcal{D}} \left[\log \pi_{\theta}(a | o) \exp \left(\frac{1}{\beta} A(o, a) \right) \right], \quad (2)$$

where A is the estimated advantage and β is a temperature. **ReinFlow** [48] makes each flow step a Gaussian transition:

$$a^{(k+1)} \sim \mathcal{N}(a^{(k)} + v_{\theta}(a^{(k)}, k) \Delta t, \sigma_{\phi}(a^{(k)}, k, o)^2 I), \quad (3)$$

which yields tractable per-step log-likelihoods for on-policy optimization such as PPO [19].

III. METHOD

A. Real-to-Sim-to-Real Whole-Body VLA Initialization

For each task, we construct a task-specific LEGS simulator and train an initial whole-body VLA that can autonomously attempt the task on hardware, providing rollouts for WM adaptation without teleoperation.

1) *Task Construction and Randomization*: Using the robot-mounted ZED 2i, we reconstruct the deployment room and objects in the LEGS simulator and define task actions, success/failure criteria, and randomization. Object position is sampled within a $0.20\text{m} \times 0.20\text{m}$ region with yaw in $[-180^\circ, 180^\circ]$; the base starts within a $0.30\text{m} \times 0.30\text{m}$ region centered 1.5 m from the table with yaw in $[-30^\circ, 30^\circ]$. Camera roll and pitch are independently perturbed within $\pm 15^\circ$.

2) *Base VLA Training*: We collect 200 synthetic episodes per task in under 30 minutes on 8 NVIDIA L40S GPUs, then fine-tune only the flow-matching action head of a task-specific GR00T N1.7 policy [6] for 50,000 steps, freezing the vision-language backbone. Training takes 3.5 hours on 8 NVIDIA H100 GPUs.

3) *Autonomous Real-Robot Rollouts*: The initial policy autonomously collects 20 real-robot rollouts per task, split evenly between successes and failures, for task-specific WM adaptation.

B. World Model Fine-Tuning

We initialize from DreamDojo’s released 2B_G1_post-train checkpoint [43] and train in two stages. LEGS simulator play data cover contact dynamics and locomotion-induced viewpoint changes; each task then uses 200 synthetic demonstrations and 20 autonomous real-robot rollouts to combine broad state-action coverage with deployment appearance.

1) *Assets*: We use over 100 household objects from open-source 3D object datasets [49]–[52] and randomize their mass and friction across episodes.

2) *Manipulation Play Data*: Each two-minute manipulation episode samples 5–8 objects with randomized table poses and one active arm; the other remains idle. We perturb the nominal grasp target with independent uniform offsets of ± 2 , ± 3 , and $\pm 2\text{cm}$ and allow up to three grasp-and-place attempts per object. All outcomes, including misses, slips, collisions, and failed placements, are retained to supervise contact dynamics.

3) *Locomotion Play Data*: Each locomotion episode samples an approach to a 0.50 m standoff followed by a random 90° turn, or an approach to 0.80 m followed by a clockwise or counterclockwise semicircle. Forward speed is 0.2–0.3 m/s, lateral velocity is zero, and SONIC tracks yaw. These trajectories cover approach, turning, and sustained walking in the deployment room.

4) *Play Data Mid-Training*: The corpus contains 400 manipulation episodes (10 hours) and 100 locomotion episodes (1 hour), totaling 500 episodes and 11 hours. Starting from 2B_G1_post-train, we mid-train for 15,000 steps over 10 hours on 8 NVIDIA H100 GPUs, producing the shared checkpoint ϕ_{mid} .

5) *Task-Specific Fine-Tuning*: For each task τ , we fine-tune ϕ_{mid} on 200 synthetic demonstrations and 20 autonomous real-robot rollouts. Synthetic trajectories cover randomized configurations and successful motions; real rollouts provide deployment-camera successes and failures that bridge the residual visual gap. Fine-tuning runs for 30,000 steps and 17 hours per task, producing ϕ_τ for policy rollout and optimization.

C. Policy Rollout within the World Model

A rollout integrates the GR00T VLA, the SONIC–MuJoCo WBC, and the DreamDojo WM. These components exchange policy commands, achieved robot states, and generated visual observations at their respective execution rates, as illustrated in Fig. 2.

1) *VLA Interface*: At policy step t , the flow-matching VLA receives an egocentric RGB observation $o_t \in \mathbb{R}^{H_i \times W_i \times 3}$, a proprioceptive state $s_t \in \mathbb{R}^{15}$, and a language instruction ℓ . It predicts a chunk of $H = 40$ whole-body targets at a 20 Hz execution rate:

$$A_t = (a_t^1, \dots, a_t^H) \sim \pi_\theta(\cdot \mid o_t, s_t, \ell) \in \mathbb{R}^{H \times 18}, \quad H = 40, \quad (4)$$

We execute the first $H_e = 24$ targets before querying the policy again, matching the execution horizon used on the real robot. Each target is represented in task space:

$$a = (\underbrace{T_L, \gamma_L}_7, \underbrace{T_R, \gamma_R}_7, \underbrace{v_x, v_y, v_\psi, h}_4) \in \mathbb{R}^{18}, \quad (5)$$

where $T_L, T_R \in SE(3)$ denote hand poses represented by position and Euler angles, $\gamma_L, \gamma_R \in [0, 1]$ denote gripper openings, (v_x, v_y, v_ψ) are planar base-velocity commands, and h is the torso-height target. The proprioceptive input uses the corresponding command-space representation:

$$s = (T_L, \gamma_L, T_R, \gamma_R, h) \in \mathbb{R}^{15}, \quad (6)$$

which contains the commanded hand poses, gripper openings, and torso height. Joint angles are not provided to the VLA.

2) *WBC State Evolution*: The SONIC–MuJoCo WBC maps each task-space target to achieved whole-body motion. SONIC computes control outputs from the current joint state, and MuJoCo advances that state. Within each target interval, this feedback loop runs at 90 Hz:

$$q_{k+1} = f(q_k, \pi_{\text{wbc}}(q_k, a)), \quad q_k \in \mathbb{R}^{43}, \quad \Delta t_{\text{wbc}} = \frac{1}{90} \text{ s}, \quad (7)$$

where q_k is the 43-dimensional joint state and f denotes the MuJoCo transition. Within the WBC, MuJoCo closes the state-feedback loop required by SONIC and supplies the achieved joint state used to condition the WM.

3) *World Model Observation Generation*: For each executed action chunk, the WM receives a seed observation and a subsampled sequence of achieved joint states, and generates $C = 12$ egocentric frames:

$$(o^{(1)}, \dots, o^{(C)}) = W_\phi(o^{(0)}, \hat{q}^{(0:C)}), \quad (8)$$

$$\hat{q}^{(m)} = \bar{q}_{\rho m}, \quad C = 12,$$

The WM is therefore conditioned on the evolved robot joint state as we mentioned above. Each generated frame is held

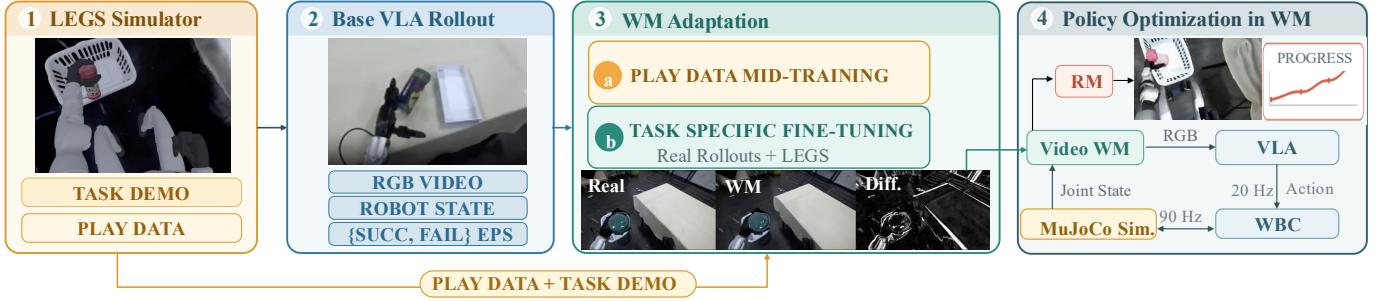


Fig. 2. **LMAO-World pipeline.** The LEGS simulator supplies synthetic task demonstrations and play data. Autonomous real-robot rollouts adapt the world model, which is coupled with the VLA and SONIC-MuJoCo WBC for policy optimization under dense SARM rewards.

for ρ policy steps, such that $o_j = o(\lceil j/\rho \rceil)$, and the terminal frame is used for the next VLA query and subsequent WM generation. This separation assigns visual prediction to the WM and robot-state evolution to the WBC, avoiding inverse-kinematics reconstruction from generated video.

Temporal alignment between the policy and WM is important because the WM is fine-tuned at 10 fps, whereas policy targets are executed at 20 Hz. We choose the stride ρ so that the state displacement between consecutive WM frames matches the displacement observed during fine-tuning:

$$\begin{aligned} \mathbb{E} \|\hat{q}^{(m+1)} - \hat{q}^{(m)}\| &= \rho \mathbb{E} \|\bar{q}_{j+1} - \bar{q}_j\| \\ &\approx \mathbb{E}_{\mathcal{D}_{\text{re}}} \|\Delta q\|, \end{aligned} \quad (9)$$

which yields $\rho = 2$. Finally, each rollout is initialized from a real observation, $o^{(0)} \sim \mathcal{D}_{\text{real}}$, anchoring generation to the deployment-domain appearance.

D. Reward Estimation on Generated Video

1) *Reward Model Training:* We train one task-specific SARM model per task, dividing each trajectory into locomotion and manipulation stages and supervising piecewise-linear progress between fixed milestones. Synthetic stage boundaries come from executed primitives; failed real rollouts follow SARM2, with progress rising to the reached stage and dropping to zero at the annotated failure. The predictor is an 8-layer, 12-head transformer ($d_{\text{model}} = 768$) over frozen DINOv3 visual and SigLIP-2 text features. It conditions on the task instruction and a 16-frame history, with up to eight rewind steps enforcing decreasing progress under backward playback. Training uses the WM rollout cadence of 10 fps with observations sampled every 0.30 s.

2) *Dense Rewards and Termination:* For a generated rollout, the model predicts progress p_t . We smooth this estimate with an exponential moving average and define the per-step reward as its increment:

$$r_t = \tilde{p}_t - \tilde{p}_{t-1}, \quad \tilde{p}_t = \alpha \tilde{p}_{t-1} + (1 - \alpha) p_t, \quad (10)$$

where $\alpha = 0.7$ suppresses frame-level prediction noise. Generated rollouts extend to $1.5\times$ the real-robot episode length to accommodate slower policies. We terminate a rollout and mark it successful when $\tilde{p}_t \geq 0.9$ for 5 consecutive inferences. Frames after this point are discarded, preventing post-completion generation from contributing an idle tail to policy training. Figure 4 visualizes this signal over a repre-

sentative WM rollout, with sampled frames linked to their corresponding progress estimates.

E. Reinforcement-Learning Instantiations

We instantiate DICE-RL, ReinFlow, and AWR within the same WM rollout and reward interface. Each optimizer generates rollouts under its own evolving policy, while the vision-language backbone remains frozen and its pooled features are cached for policy and value updates.

1) *DICE-RL:* We freeze the GR00T flow policy and train the residual actor in Eq. (1) together with a 20-member critic ensemble. Because dense progress rewards remain noisy at the action-chunk level, we add the discounted Monte Carlo return to the temporal-difference target:

$$y_t = r_t + \gamma(1 - d_t) \min_j Q_j^- + w R_t^{\text{mc}}, \quad w = 2, \quad (11)$$

where $R_t^{\text{mc}} = \sum_{k \geq t} \gamma^{k-t} r_k$ is computed over the truncated generated rollout. The bootstrap preserves local credit assignment, while the Monte Carlo term anchors the critic to the rollout-level outcome. We clip negative progress increments to zero and regularize the residual magnitude. Each actor update evaluates $K = 8$ noise-conditioned action chunks. We use $\gamma = 0.995$, batch size 128, and actor and critic learning rates of 10^{-4} .

2) *ReinFlow:* Unlike DICE-RL, ReinFlow updates the flow policy itself. We fully fine-tune the VLA’s flow-matching action head, while keeping the vision-language backbone frozen. The Gaussian transitions in Eq. (3) provide tractable per-step likelihoods for a clipped PPO objective. We additionally anchor the fine-tuned action to the frozen base policy and penalize changes in its temporal differences:

$$\begin{aligned} \mathcal{L}_{\text{RF}}(\theta) &= -\mathbb{E} \left[\min \left(\rho \hat{A}, \text{clip}(\rho, 1 - \varepsilon, 1 + \varepsilon) \hat{A} \right) \right] \\ &\quad + c_v \mathcal{L}_V + \beta_{\text{BC}} \mathbb{E} [\|a_\theta - \bar{a}\|_2^2 + \|\delta a_\theta - \delta \bar{a}\|_2^2], \end{aligned} \quad (12)$$

where $\rho = \pi_\theta(a | o) / \pi_{\theta_{\text{old}}}(a | o)$, $\hat{A}$ is estimated by the state-value critic, $\bar{a}$ is the frozen-policy action, and $\delta a_t = a_{t+1} - a_t$. We warm-start the critic for 3,000 steps before updating the action head.

3) *AWR:* We use the advantage-conditioned AWR formulation of $\pi_{0.6}^*$ [15], which replaces likelihood-weighted regres-

sion with conditioning on a binary improvement variable:

$$p(I \mid A^{\pi_{\text{ref}}}(o, a, \ell)) = \delta(A^{\pi_{\text{ref}}}(o, a, \ell) > \epsilon_{\ell}),$$

$$\hat{\pi}(a \mid o, \ell) \propto \pi_{\text{ref}}(a \mid o, \ell) \left[\frac{\pi_{\text{ref}}(a \mid I, o, \ell)}{\pi_{\text{ref}}(a \mid o, \ell)} \right]^{\beta}. \quad (13)$$

We estimate $A^{\pi_{\text{ref}}} = r_t + \gamma V(o_{t+1}) - V(o_t)$, set $\epsilon_{\ell} = 0$ and $\beta = 1$, and append the resulting label to the language input. The policy is trained on both positive and negative chunks with the original flow-matching objective and conditioned on improvement at deployment.

F. Closed-Loop Policy Improvement

1) *Round-Based Optimization*: After task-specific fine-tuning, we freeze the WM and optimize each task for 15 rounds. In each round, the current policy generates 32 episodes initialized from frames sampled from the real rollouts used for WM fine-tuning. Episodes run for up to 60 s of WM time and are retained regardless of outcome. The progress model labels each video and truncates successful episodes at the success crossing (Sec. III-D), removing post-completion idle frames.

The policy is trained for 1×10^3 steps per round, carrying its policy, value or critic, target-network, and optimizer states across rounds while refreshing data under the current policy. Each task therefore uses 480 WM-generated episodes and 1.5×10^4 updates. We then evaluate each checkpoint on real robot.

2) *Compute*: Each run uses one node with 8 NVIDIA H100 GPUs. Eight workers generate rollouts in parallel with independent WM replicas, WBC containers, and MuJoCo instances; the same GPUs then perform RL updates. A 15-round run takes about 12 hours: 75% for autoregressive video generation, 20% for RL updates, and the remainder for progress labeling and feature caching. Video generation is the primary bottleneck.

IV. EXPERIMENTS

We ask three research questions in this section:

- **Q1**: How do simulated whole-body play data affect WM fidelity and downstream policy learning?
- **Q2**: How does a world model improve VLA post-training?
- **Q3**: Which aspects make our pipeline particularly effective for humanoid loco-manipulation?

A. Experimental Setup

1) *Tasks*: We evaluate on three real-robot tasks that require increasingly varied coordination between locomotion and manipulation (Fig. 3). In *Pick-and-Place Bottle* (PNP), the robot walks to a table, grasps a bottle with its left hand, and places it upright in a box. *Pick-Turn-Place Bottle* (PTP) adds post-grasp locomotion: after lifting the bottle, the robot steps back, turns approximately 90°, approaches a floor basket, squats, and places the bottle inside. In *Bimanual Move Basket* (Basket), the robot turns and walks toward a side platform, squats and lifts a basket bimanually, turns back toward the table, then walk and places the basket on it. Together, these tasks cover manipulation from a stationary base, object transport through locomotion, and bimanual whole-body control.

2) *World Model Evaluation*: For each task, we compare two WMs under a matched budget of 45,000 optimization steps: the full model receives 15,000 steps of mid-training on play data followed by 30,000 task-specific steps, while the ablation skips play data and uses all 45,000 steps for task-specific fine-tuning. Both train on the same 200 synthetic episodes and 20 autonomous real-robot rollouts, comprising 10 successful and 10 failed trajectories. Evaluation uses a disjoint set of 20 synthetic and 20 real episodes per task, with 10 successful and 10 failed trajectories from each source. We replay each recorded action trajectory and compare the generated frames with the temporally aligned ground truth, excluding terminal-frame padding. We report PSNR, interaction PSNR (iPSNR), SSIM, LPIPS, and FID. iPSNR is computed over the union of robot-hand and manipulated-object masks obtained with Grounding DINO [53] and SAM 2 [54]. The same frames and masks are used for both models; higher PSNR, iPSNR, and SSIM are better, while lower LPIPS and FID are better.

3) *Policy Evaluation*: For each policy-task pair, we report success rate (SR) over 20 real-robot trials. Success requires the bottle to stand upright in the tabletop box for Pick-and-Place Bottle, the bottle to be placed in the floor basket for Pick-Turn-Place Bottle, or the bimanually carried basket to be placed stably on the table for Bimanual Move Basket. To track learning progress, we also evaluate intermediate policies on the real robot every two rounds and in the LEGS simulator and WM rollouts (Fig. 3). Success in the LEGS simulator is determined by full-state task predicates, whereas success in WM rollouts is determined by the progress reward model.

4) *Baselines and Ablations*: All methods start from the same synthetic-data BC checkpoint. **BC** receives no RL post-training. **Real RL** applies DICE-RL to a budget of 20 autonomous real-robot rollouts, matching the real-rollout budget used for task-specific WM fine-tuning, while **Sim RL** applies DICE-RL directly in the LEGS simulator. Among WM-based baselines, **VLAW** [29] behavior-clones successful WM rollouts, whereas **WMPO** [30] uses GRPO with sparse terminal rewards. Because our VLA has a flow-matching action head, we implement WMPO with Flow-GRPO [55]. We instantiate our pipeline with AWR, ReinFlow, and DICE-RL. All ablations use DICE-RL, the best-performing of these three RL algorithms. **Sparse Reward** replaces dense progress with terminal success feedback. **Offline** uses a fixed set of 480 WM rollouts (15 rounds $\times$ 32 episodes), generated once by the base BC policy, matching the total rollout budget without refreshing data as the policy changes. **Without play data** omits WM mid-training on play data.

B. Q1: Effect of WM Mid-Training on Play Data

Table I shows a consistent benefit from play data: the play-data model outperforms the base initialization on every metric across all three tasks. Both global PSNR and interaction-region iPSNR improve, with the iPSNR gain exceeding the corresponding PSNR gain for every task. Because iPSNR focuses on the robot hands and manipulated object, this larger improvement suggests that play data particularly strengthens

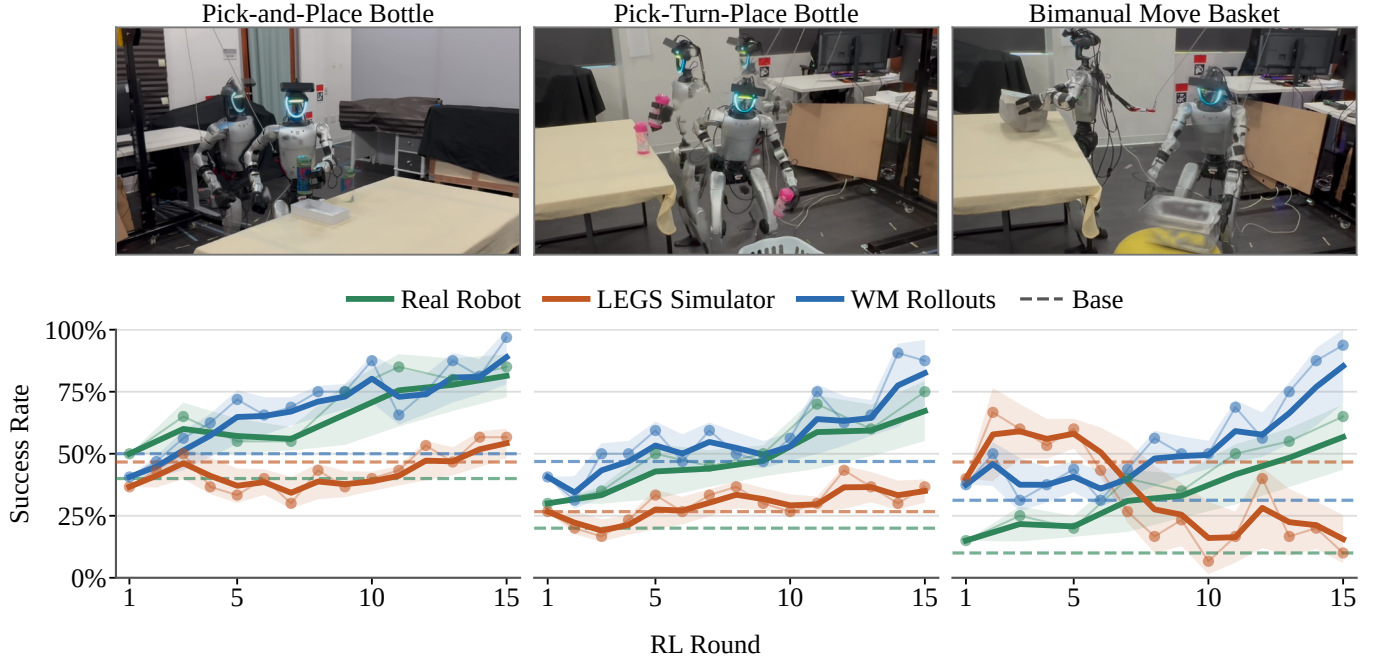


Fig. 3. **DICE-RL improvement across three loco-manipulation tasks.** Top: representative real-robot executions. Bottom: success over 15 RL rounds on the real robot, in the LEGS simulator, and in world model rollouts. Solid lines are exponential moving averages, faint lines and markers are per-round measurements, bands are exponentially weighted standard deviations, and dashed lines show base-BC success.

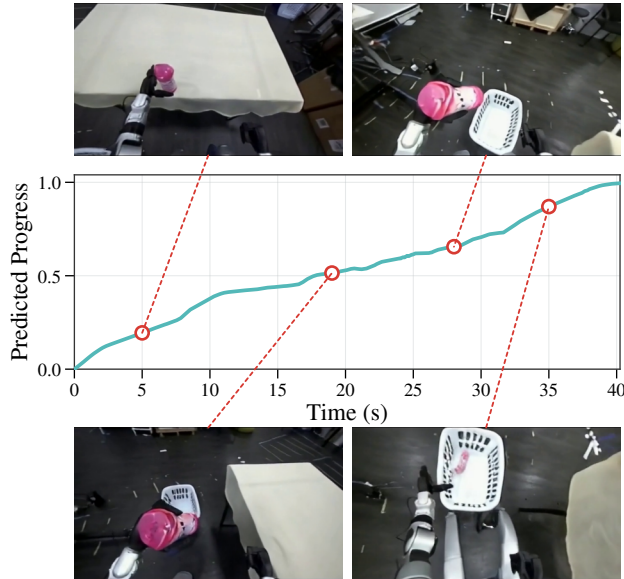


Fig. 4. **Dense progress on a world model rollout.** Red circles link four **WM-generated** views to their progress estimates. Increments of the smoothed progress prediction provide dense rewards.

the modeling of contact-rich physical interactions and object dynamics, rather than merely improving global scene appearance.

The policy ablation in Table II shows a corresponding downstream effect. Removing play data from the DICE-RL pipeline reduces SR on all three tasks. The mean success rate falls from 75.0% to 63.3%. Play data mid-training therefore improves both WM fidelity and downstream policy optimization.

TABLE I
WORLD MODEL FIDELITY WITH AND WITHOUT PLAY DATA. BASE USES 45,000 TASK-SPECIFIC STEPS; PLAY USES 15,000 PLAY DATA MID-TRAINING STEPS FOLLOWED BY 30,000 TASK-SPECIFIC STEPS. BOTH USE IDENTICAL TASK DATA AND HELD-OUT EPISODES; BEST VALUES ARE BOLD.

Task	Variant	PSNR $\uparrow$	iPSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FID $\downarrow$
PNP	Base	20.22	15.12	0.75	0.24	32.61
	Play	21.46	16.71	0.77	0.23	30.97
PTP	Base	18.38	14.96	0.70	0.28	33.08
	Play	19.08	15.86	0.71	0.28	31.97
Basket	Base	17.70	13.06	0.67	0.30	37.54
	Play	18.55	14.07	0.68	0.29	37.42

C. Q2: WM for Policy Learning

Table II shows that policy learning within the WM is effective across all three RL algorithms. AWR, ReinFlow, and DICE-RL all achieve higher mean real robot SR than BC, Sim RL, and Real RL. DICE-RL performs best overall, reaching 85%, 75%, and 65% success across the three tasks. Real RL is constrained by interaction efficiency: its budget of 20 real-robot rollouts provides limited opportunities for exploration and policy refinement. The adapted WM turns the same real-rollout budget into 480 on policy rollouts without further hardware interaction, demonstrating the value of scalable WM experience.

Sim RL reaches only 30% on PNP and fails all trials on other two tasks, underperforming BC despite using the same LEGS simulator. Direct RL against the simulator can amplify residual sim-to-real gap, leading the policy to exploit

TABLE II
REAL-ROBOT POLICY SUCCESS RATE OVER 20 TRIALS. ABLATIONS USE DICE-RL; BEST RESULTS ARE BOLD.

Baselines				RL Algorithms			
Method	PNP	PTP	Basket	Method	PNP	PTP	Basket
BC	40%	20%	10%	AWR	75%	70%	70%
Real RL	60%	45%	40%	ReinFlow	60%	50%	45%
Sim RL	30%	0%	0%	DICE-RL	85%	75%	65%
VLAW	45%	20%	35%	Ablations			
WMPO	35%	30%	35%	Sparse	65%	40%	30%
				Offline	70%	55%	50%
				w/o Play	70%	65%	55%

simulator-specific behaviors. The learning curves in Fig. 3 provide complementary evidence: WM and real-robot success rates track one another and rise over optimization, whereas success in the LEGS simulator remains nearly level on PNP and PTP, and decreases substantially on Basket. Thus, the adapted WM provides a substantially better proxy for real policy improvement than the LEGS simulator.

D. Q3: Design Choices for Loco-Manipulation

VLAW highlights the importance of RL optimization over the full rollout distribution. It behavior-clones only successful WM rollouts and averages 33.3% success, whereas DICE-RL directly optimizes the policy using successful, failed, and suboptimal episodes and reaches 75.0%. Dense progress provides graded supervision without task completion, allowing non-successful experience to improve the policy.

WMPO instead highlights the importance of dense, sample-efficient credit assignment. It optimizes the policy with sparse terminal feedback and also averages 33.3%. Because autoregressive WM rollouts are four to eight times slower than LEGS simulator rollouts, each trajectory must be used efficiently. Replacing dense rewards with sparse feedback lowers mean success from 75.0% to 45.0%, while a fixed offline rollout set yields 58.3% and removing play data yields 63.3%. These results support dense rewards, policy-dependent data refresh, and play data.

The pipeline is not tied to one optimizer. AWR and DICE-RL achieve comparable mean success rates of 71.7% and 75.0%, while ReinFlow is less stable but still improves over BC on every task. This consistency shows that the pipeline supports multiple RL algorithms for whole-body VLA post-training.

V. CONCLUSION

We presented LMAO-World, a teleoperation-free sim-to-dream-to-real pipeline for RL post-training of whole-body humanoid VLAs. Simulated play data broaden WM coverage, autonomous real-robot rollouts adapt the WM to deployment observations, and dense progress rewards enable learning from successful and unsuccessful generated trajectories. Across three tasks, play data improves all evaluated WM metrics and downstream policy success; DICE-RL and AWR achieve mean success rates of 75.0% and 71.7%. The results identify WM

adaptation, dense rewards, policy-dependent rollout refresh, and play data as key to effective loco-manipulation post-training.

Current limitations include task-specific WM and reward adaptation, single-view observations, autoregressive drift, and costly video generation. Although target-task data require no teleoperation, the base WM inherits human-video pretraining. Future work should improve long-horizon and multi-view prediction, accelerate rollouts, and continually update the policy and WM from autonomous experience.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis *et al.*, “3d gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [6] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, “Gr00t n1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [7] Z. Luo, Y. Yuan, T. Wang, C. Li, F. Castañeda, S. Chen, Z.-A. Cao, J. Li, D. Minor, Q. Ben *et al.*, “Sonic: Supersizing motion tracking for natural humanoid whole-body control,” *Science Robotics*, vol. 11, no. 117, p. ead4592, 2026.
- [8] Y. Ze, S. Zhao, W. Wang, A. Kanazawa, R. Duan, P. Abbeel, G. Shi, J. Wu, and C. K. Liu, “Twist2: Scalable, portable, and holistic humanoid data collection system,” *arXiv preprint arXiv:2511.02832*, 2025.
- [9] H. Kim, T. Chen, J. Sun, L. W. Osterberg, Q. Chen, K. Wang, and M. Schwaiger, “Legs: Fine-tuning teleop-free vlars for humanoid loco-manipulation in an embodied gaussian splatting world,” *arXiv preprint arXiv:2606.01458*, 2026.
- [10] Y.-J. Wang, J. Li, S. Chen, T. E. Truong, P. Xu, P. Abbeel, R. Duan, K. Sreenath, A. Kanazawa, C. Sferrazza *et al.*, “Vlk: Learning humanoid loco-manipulation from synthetic interactions in reconstructed scenes,” *arXiv preprint arXiv:2606.30645*, 2026.
- [11] H. Hu, S. Mirchandani, and D. Sadigh, “Imitation bootstrapped reinforcement learning,” *arXiv preprint arXiv:2311.02198*, 2023.
- [12] J. Yang, M. S. Mark, B. Vu, A. Sharma, J. Bohg, and C. Finn, “Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 4804–4811.
- [13] S. Huang, Z. Zhang, T. Liang, Y. Xu, Z. Kou, C. Lu, G. Xu, Z. Xue, and H. Xu, “Mentor: Mixture-of-experts network with task-oriented perturbation for visual reinforcement learning,” *arXiv preprint arXiv:2410.14972*, 2024.
- [14] P. Wu, Y. Shentu, Q. Liao, D. Jin, M. Guo, K. Sreenath, X. Lin, and P. Abbeel, “Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation,” *arXiv preprint arXiv:2503.07771*, 2025.
- [15] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia *et al.*, “ $\pi_{0.6}^*$: A vla that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [16] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.

- [17] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, "Confrt: A reinforced fine-tuning method for vla models via consistency policy," *arXiv preprint arXiv:2502.05450*, 2025.
- [18] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu, "RL-100: Performant robotic manipulation with real-world reinforcement learning," *arXiv preprint arXiv:2510.14830*, 2025.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [20] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering your diffusion policy with latent space reinforcement learning," *arXiv preprint arXiv:2506.15799*, 2025.
- [21] H. Niu, Q. Chen, T. Liu, J. Li, G. Zhou, Y. Zhang, J. Hu, and X. Zhan, "xtd: Cross-domain adaptation via diffusion-based trajectory editing," *arXiv preprint arXiv:2409.08687*, 2024.
- [22] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan *et al.*, "Self-improving vision-language-action models with data generation via residual rl," *arXiv preprint arXiv:2511.00091*, 2025.
- [23] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, "Residual off-policy rl for finetuning behavior cloning policies," *arXiv preprint arXiv:2509.19301*, 2025.
- [24] Z. Sun and S. Song, "From prior to pro: Efficient skill mastery via distribution contractive rl finetuning," *arXiv preprint arXiv:2603.10263*, 2026.
- [25] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations (ICLR)*, 2020.
- [26] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel, "Daydreamer: World models for physical robot learning," in *Conference on Robot Learning (CoRL)*, 2022.
- [27] L. Barcellona, A. Zadaianchuk, D. Allegro, S. Papa, S. Ghidoni, and E. Gavves, "Dream to manipulate: Compositional world models empowering robot imitation learning with imagination," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 56 729–56 763.
- [28] A. L. Chandra, I. Nematollahi, C. Huang, T. Welschhold, W. Burgard, and A. Valada, "Diwa: Diffusion policy adaptation with world models," *Conference on Robot Learning (CoRL)*, 2025.
- [29] Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn, "Vlaw: Iterative co-improvement of vision-language-action policy and world model," *arXiv preprint arXiv:2602.12063*, 2026.
- [30] F. Zhu, Z. Yan, Z. Hong, Q. Shou, X. Ma, and S. Guo, "Wmpo: World model-based policy optimization for vision-language-action models," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 62 486–62 502.
- [31] Z. Jiang, K. Liu, Y. Qin, S. Tian, Y. Zheng, M. Zhou, C. Yu, H. Li, and D. Zhao, "World4rl: Diffusion world models for policy refinement with reinforcement learning for robotic manipulation," *arXiv preprint arXiv:2509.19080*, 2025.
- [32] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen *et al.*, "Rise: Self-improving robot policy with compositional world model," *arXiv preprint arXiv:2602.11075*, 2026.
- [33] A. K. Sharma, Y. Sun, N. Lu, Y. Zhang, J. Liu, and S. Yang, "Worldgymnast: Training robots with reinforcement learning in a world model," *arXiv preprint arXiv:2602.02454*, 2026.
- [34] X. Liu, Z. Bai, H. Ci, K. Y. Ma, and M. Z. Shou, "World-vla-loop: Closed-loop learning of video world model and vla policy," *arXiv preprint arXiv:2602.06508*, 2026.
- [35] Z. Jiang, S. Zhou, Y. Jiang, Z. Huang, M. Wei, Y. Chen, T. Zhou, Z. Guo, H. Lin, Q. Zhang *et al.*, "Wovr: World models as reliable simulators for post-training vla policies with rl," *arXiv preprint arXiv:2602.13977*, 2026.
- [36] Y. Xue, L. Xu, Z. Liu, Z. Wu, Z. Gu, X. Song, B. Jia, and Z. Wang, "Worldsample: Closed-loop real-robot rl with world modelling," *arXiv preprint arXiv:2607.02431*, 2026.
- [37] T. Yin, Z. Mei, Z. Zheng, M. Yamane, D. Wang, J. Sceats, S. M. Bateman, L. Zha, A. Badithela, O. Shorinwa, and A. Majumdar, "Playworld: Learning robot world models from autonomous play," *arXiv preprint arXiv:2603.09030*, 2026.
- [38] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li *et al.*, "Sam 3d: 3dfy anything in images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 7220–7232.
- [39] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.
- [40] Q. Chen, J. Yu, M. Schwager, P. Abbeel, F. Shentu, and P. Wu, "Sarm: Stage-aware reward modeling for long horizon robot manipulation," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 17 091–17 116.
- [41] M. Tschannen, A. Gritsenko, X. Wang, M. F. Naeem, I. Alabdulmohsin, N. Parthasarathy, T. Evans, L. Beyer, Y. Xia, B. Mustafa *et al.*, "Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features," *arXiv preprint arXiv:2502.14786*, 2025.
- [42] Q. Chen, H. Zheng, J. Yu, S. Huang, J. Sun, K. Goldberg, C. Wen, P. Abbeel, Y. Shentu, P. Wu *et al.*, "Sarm2: Multi-task stage aware reward modeling for self improving robotic manipulation," *arXiv preprint arXiv:2606.10305*, 2026.
- [43] S. Gao, W. Liang, K. Zheng, A. Malik, S. Ye, S. Yu, W.-C. Tseng, Y. Dong, K. Mo, C.-H. Lin *et al.*, "Dreamdojo: A generalist robot world model from large-scale human videos," *arXiv preprint arXiv:2602.06949*, 2026.
- [44] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding *et al.*, "Cosmos world foundation model platform for physical ai," *arXiv preprint arXiv:2501.03575*, 2025.
- [45] B. Chen, D. Martí Monsó, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann, "Diffusion forcing: Next-token prediction meets full-sequence diffusion," *Advances in Neural Information Processing Systems*, vol. 37, pp. 24 081–24 125, 2024.
- [46] X. Huang, Z. Li, G. He, M. Zhou, and E. Shechtman, "Self forcing: Bridging the train-test gap in autoregressive video diffusion," *Advances in Neural Information Processing Systems*, vol. 38, pp. 167 283–167 308, 2026.
- [47] A. K. Jain, Y. Wu, J. Farebrother, G. Swamy, and A. Bajcsy, "Weaver, better, faster, longer: An effective world model for robotic manipulation," *arXiv preprint arXiv:2606.13672*, 2026.
- [48] T. Zhang, C. Yu, S. Su, and Y. Wang, "Reinflow: Fine-tuning flow matching policy with online reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 38, pp. 106 282–106 319, 2026.
- [49] B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar, "Benchmarking in manipulation research: The YCB object and model set and benchmarking protocols," *IEEE Robotics & Automation Magazine*, vol. 22, no. 3, pp. 36–52, 2015.
- [50] M. Deitke, R. Liu, M. Wallingford, H. Ngo, O. Michel, A. Kusupati, A. Fan, C. Laforte, V. Voleti, S. Y. Gadre *et al.*, "Objaverse-xl: A universe of 10m+ 3d objects," *Advances in Neural Information Processing Systems*, vol. 36, pp. 35 799–35 813, 2023.
- [51] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, "Google scanned objects: A high-quality dataset of 3d scanned household items," in *2022 international conference on robotics and automation (ICRA)*. Ieee, 2022, pp. 2553–2560.
- [52] J. Collins, S. Goel, K. Deng, A. Luthra, L. Xu, E. Gundogdu, X. Zhang, T. F. Y. Vicente, T. Dideriksen, H. Arora *et al.*, "Abo: Dataset and benchmarks for real-world 3d object understanding," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 21 094–21 104.
- [53] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *European Conference on Computer Vision*, 2024, pp. 38–55.
- [54] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, "SAM 2: Segment anything in images and videos," in *International Conference on Learning Representations*, 2025.
- [55] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang, "Flow-GRPO: Training flow matching models via online reinforcement learning," *arXiv preprint arXiv:2505.05470*, 2025.